

Discovery Executive Summary

Project: discovery-test · Generated: 11/08/2026, 16:04:10

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a monorepo combining a Laravel 12 PHP backend, a React 19 TypeScript SPA frontend, and a Node.js/Express dev-API shim — all serving a voice observability product with two business domains: **Discovery** (IVR mapping) and **Connect** (TFN reachability monitoring). The overall architecture is immature: controllers perform direct Eloquent ORM access, embed business logic (reachability calculations, IVR tree building), and duplicate code across files, while no Repository layer exists. The most severe hotspots are a fat `LegacyReportController` combining DB queries, KPI math, and response formatting; a `buildTree()` function copy-pasted verbatim into two controllers; and a reachability formula that appears three times (two PHP controllers, one service). On the frontend, `LegacyMonitorPoller.jsx` is a class component with an interval memory leak, and `LegacyDashboardWidget.tsx` mixes multi-domain data fetching with rendering, bypassing the TanStack Query pattern used elsewhere. The dominant systemic risk is **change amplification**: any change to the reachability threshold (currently hardcoded at 90%) or IVR tree shape must be applied in multiple controllers, a service, and a Node.js route handler — with no test coverage to catch a missed update.

\$1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	~74 LOC avg	Good
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	13 direct Eloquent calls across 4 controllers	Moderate
H3	Missing Repository Pattern	Direct DB access outside repos	<10	10–20	>20	23+ Eloquent static calls; 0 repositories exist	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files with business logic	0	1–5	>5	1 (LegacyDataMapper using unsafe extract())	Moderate
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% — all queries via Eloquent	Good
H7	God Classes	Files >1000 LOC	0	1–3	>3	0	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	6 (LegacyReportController, RealTimeTestService, DashboardController)	High Risk
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	0%	Good
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	~120 LOC avg	Good
F2	Missing Frontend Service Layer	Components with inline API calls	<10	10–20	>20	6 components with direct api.get/post	Moderate

F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	Good
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	1 level	Good
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (<code>LegacyMonitorPoller.jsx</code> , <code>LegacyDashboardWidget.tsx</code>)	Moderate
H10	Duplicate Business Logic (<i>additional</i>)	Copy-pasted algorithm occurrences	0	1–2	>2	3 (<code>buildTree()</code> ×2 controllers + JS; reachability ×2 controllers + service)	High Risk

\$1.4 Actions Required

Hotspot	Action	Rating	Priority
H3 Missing Repository Pattern	Create <code>DiscoveryJobRepository</code> and <code>ConnectMonitorRepository</code> ; bind interfaces in <code>AppServiceProvider</code> ; update <code>RealTimeTestService</code> for constructor DI	High Risk	Critical
H8 Domain Boundary Violations	Split <code>RealTimeTestService</code> into domain-specific orchestrators; route <code>LegacyReportController</code> cross-domain reads through <code>DiscoveryService</code> ; create <code>DashboardService</code>	High Risk	Critical
H10 Duplicate Business Logic	Extract <code>buildTree()</code> into <code>DiscoveryService</code> ; extract reachability formula into <code>ConnectService::computeReachability()</code> ; delete private controller copies	High Risk	Critical
H2 Missing Service Layer	Create <code>ConnectService</code> and <code>DashboardService</code> ; thin controllers to single-line service delegation	Moderate	High
H5 Shared Utility Abuse	Delete <code>LegacyDataMapper.php</code> ; replace <code>extract()</code> calls with explicit destructuring; add Form Request validation	Moderate	High
F5 Legacy Component Patterns	Convert <code>LegacyMonitorPoller.jsx</code> to TypeScript function component with <code>useEffect</code> cleanup; add <code>ErrorBoundary.tsx</code>	Moderate	High
F2 Missing Frontend Service Layer	Create <code>connectApi.ts</code> and <code>discoveryApi.ts</code> ; migrate <code>LegacyDashboardWidget.tsx</code> to <code>useQuery</code> ; centralize all endpoint paths	Moderate	Medium

\$1.5 Expected Outcomes

- Testability:** Repository interfaces allow `RealTimeTestService` , `ConnectController` , and `DashboardController` to be unit-tested with in-memory fakes instead of live databases; the reachability formula and IVR tree builder gain straightforward coverage.
- Change isolation:** Splitting `RealTimeTestService` by bounded context means a Discovery schema change does not require touching any Connect file, shrinking blast radius for routine feature work.
- Single source of truth:** Extracting `buildTree()` and the reachability formula into dedicated service methods eliminates three-location duplication; the 90% alert threshold becomes a named constant changed in exactly one place.
- Frontend reliability:** Adding `ErrorBoundary.tsx` and fixing the `LegacyMonitorPoller` interval leak prevents full-page crashes on API failure and stops memory accumulation during navigation.
- Architectural readiness:** Bounded-context separation with ACLs between Discovery and Connect prepares the platform for eventual microservice extraction — either module can be independently deployed once its data access is fully encapsulated behind a service interface.

The full report (34 KB, including all code evidence and Mermaid diagrams) is saved to `docs/discovery/01-architecture-design.md` . The orchestration UI will convert it to PDF automatically.,"stop_reason":"end_turn","session_id":"7a812e19-13a2-40bc-8fd8-11d9e79989f8","total_cost_usd":1.8304448000000004,"usage":{"input_tokens":5,"cache_creation_input_tokens":28476,"cache_read_input_tokens":99390,"output_tokens":4450,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":28476,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":{"input_tokens":1,"output_tokens":2306,"cache_read_input_tokens":43454,"cache_creation_input_tokens":2135,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":2135},"type":"message"},"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":10502,"outputTokens":15,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.010577,"contextWindow":200000,"maxOutputTokens":32000},"c-sonnet-4-6":{"inputTokens":46,"outputTokens":35734,"cacheReadInputTokens":2440596,"cacheCreationInputTokens":129992,"webSearchRequests":0,"costUSD":1.8198678000000004,"contextWindow":200000,"maxOutputTokens":32000},"terminal_reason":"completed","fast_mode_state":"off","uuid":"3f79d4f6-72b5-419b-a4c6-dfbf21bb9356"}